

ContractGuard

Testing Documentation

2026/04/30

Students:	Chunyang Wang	21115214
	Justin Siak	22449184

Supervisor:	Dr. Graham Healy
--------------------	-------------------------

Contents

Contents.....	2
1. Overview.....	2
2. Overall Testing Approach.....	2
3. Automated Testing During Development.....	3
3.1 Backend Unit and Integration Tests.....	3
3.2 Parser Validation.....	4
3.3 Generated Test Execution.....	5
4. Pipeline, CLI, and Manual Testing.....	5
4.1 Full Real World API Specification Testing.....	5
4.2 GitHub Actions Integration Testing with External Developer.....	7
4.3 Project CI/CD Testing in GitLab.....	8
4.4 Manual CLI and Smoke Testing.....	9
4.5 Frontend Manual Testing.....	10
5. LLM Evaluation During Development.....	15
5.1 What the Archived Runs Show.....	16
5.2 Suggested-Test Generation.....	17
5.3 Benefits, Struggles, and Accommodations.....	17
6. Summary of What Was Tested.....	18
7. Limitations and Remaining Gaps.....	19

1. Overview

Testing in ContractGuard developed in layers alongside the product itself. The project is not just a frontend with a few API calls behind it; it is a workflow that parses specifications, generates tests, executes them against live APIs, stores results, and then uses an LLM to help interpret failures. Because of that, the testing approach had to cover both conventional software behavior and the more awkward parts of the system, such as auth-sensitive endpoints, external API variation, and non-deterministic model output.

In practice, this led to a mixed strategy. Stable backend logic was covered with ordinary automated tests. The parser and execution pipeline were checked through scripts and CI stages. Real API behavior was explored through generated suites and CLI runs. The LLM layer was treated separately again, because what mattered there was not only whether it produced text, but whether that text was grounded in the failure evidence and useful for the next debugging step.

This report summarises that testing process, what it gave us, and where the limits still are.

2. Overall Testing Approach

The most useful part of the testing strategy was that it matched the structure of the system rather than forcing everything into one style of test.

1. Parser validation was used to confirm that OpenAPI inputs could be turned into the internal representation used throughout the rest of the pipeline.

2. Backend unit and integration tests were used to check request handling, persistence, provider clients, LLM facing endpoints, and full explanation or suggested test flows.
3. Generated suite execution was used to check that tests produced from a specification could actually run against real endpoints and record meaningful results.
4. Pipeline testing was used to make sure the parser, IR generation, test generation, and test runner stages linked together cleanly.
5. LLM benchmarking was used to assess explanation quality and followup test generation, rather than pretending that model output should be judged in the same way as a normal endpoint handler.

There were also some recurring difficulties. External APIs do not always fail in a neat way, especially when authentication is involved. A negative test may surface a 401 before it ever reaches the behavior the test writer was trying to probe. The LLM side had a different problem: even when a response looked fluent, it still needed to be checked for grounding, usefulness, and whether it stayed close to the actual evidence bundle. The project accommodated those issues by using mocks where determinism mattered, in-memory database setup for backend isolation, explicit auth workflows for CLI runs, and archived LLM evaluation runs with score summaries and saved outputs.

3. Automated Testing During Development

3.1 Backend Unit and Integration Tests

The backend is the most conventionally tested part of the system. There are 11 Python test modules under `src/backend/tests`, covering provider clients, LLM endpoints, logistics and run history endpoints, settings management, merge behavior, latest run lookup, and the full failure explanation and suggested test flows.

Some of the strongest examples are:

- `test_provider_clients.py`, which checks provider response parsing, missing-key handling, model listing, and structured handling of rate limit and quota errors.
- `test_failure_explanation_flow.py`, which checks valid JSON responses, invalid JSON retries, empty output failure handling, and hard fail behavior when a model returns rate limit errors.
- `test_llm_suggested_test_flow.py`, which checks the logic around deterministic follow-up test suggestions and the shape of model generated suggestions.
- `test_logistics_endpoints.py`, which uses an in-memory SQLite database to test persistence heavy behavior without polluting later test cases.

This part of the test suite is where the project gets its fastest feedback loop. It is also where the main accommodations for reliability appear most clearly: external provider calls are mocked, and database state is isolated instead of shared.

While preparing this report, the backend test suite was rerun locally with:

```
python -m unittest discover -s src/backend/tests -p 'test_*.py'
```

The result was:

- Ran 80 tests in 0.320s
- OK

That result does not prove that every runtime scenario is covered, but it does show that the core backend checks are currently stable.

3.2 Parser Validation

The parsing stage is checked with `src/test_parser.py`, which loads a sample Petstore specification and prints the parsed title, version, number of endpoints, and a first endpoint example. This is better described as a smoke check than a full assertion heavy unit test, but it still plays an important role because parsing is the first step in the whole workflow. If parsing is wrong, every later stage is built on bad data.

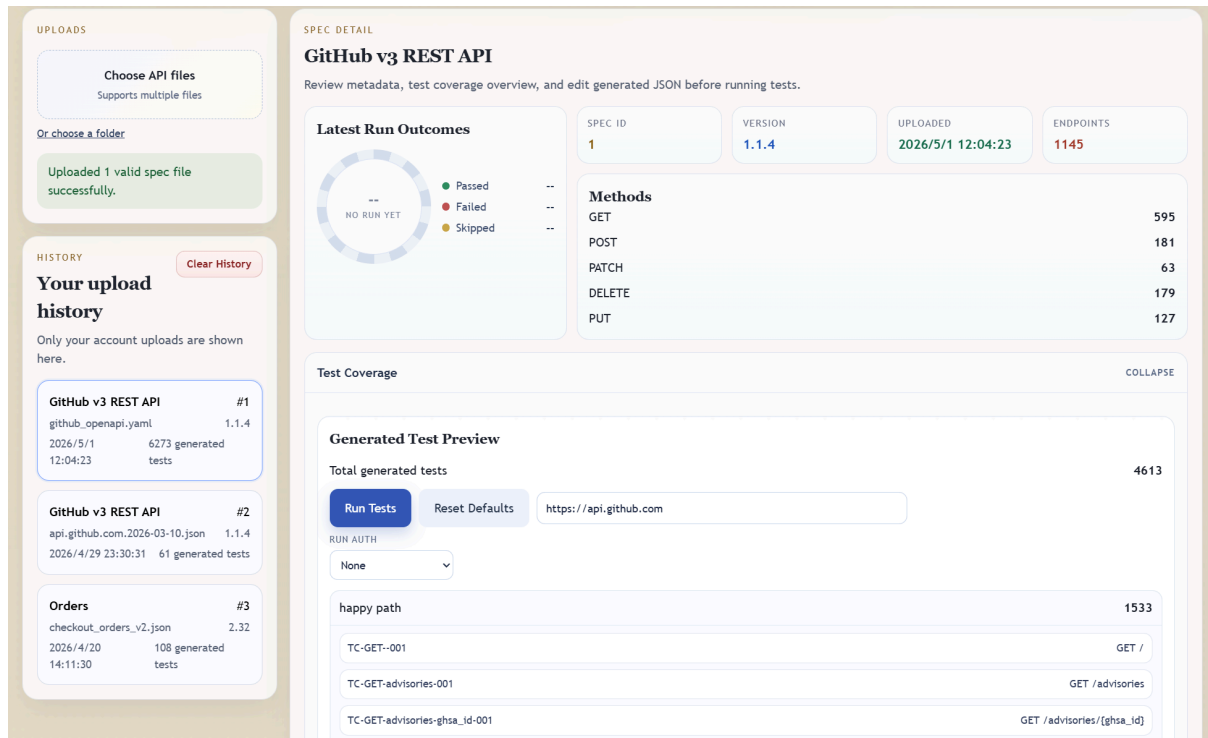
```
None  
python test_parser.py
```

The output showed:

- Title: Swagger Petstore - OpenAPI 3.0
- Version: 1.0.27
- Total endpoints: 19
- First endpoint example: PUT /pet

The benefit of this check is simplicity. The limitation is that it is still a smoke script rather than a richer assertion based parser suite.

The parser and generator were also tested with a much larger GitHub REST API specification. ContractGuard was able to parse the specification and generate more than 6,000 test cases. The frontend was able to display the generated cases correctly, but full execution became time consuming and was manually stopped after 30 minutes. This shows that the parsing and generation workflow can scale to large specifications, while execution time remains a practical limitation.



Successfully parsed GitHub REST API (Generated over 6,000 test cases).

3.3 Generated Test Execution

The test runner in `src/test_runner/run_test.py` is the bridge between generated cases and real API behavior. It loads JSON suites, resolves auth from CLI arguments or environment variables, redacts sensitive headers, executes requests, and records outcomes as PASS, FAIL, or SKIP.

This part matters because ContractGuard is ultimately meant to test APIs, not just its own internal helper functions. The runner is therefore one of the most important pieces of the overall quality story.

4. Pipeline, CLI, and Manual Testing

4.1 Full Real World API Specification Testing

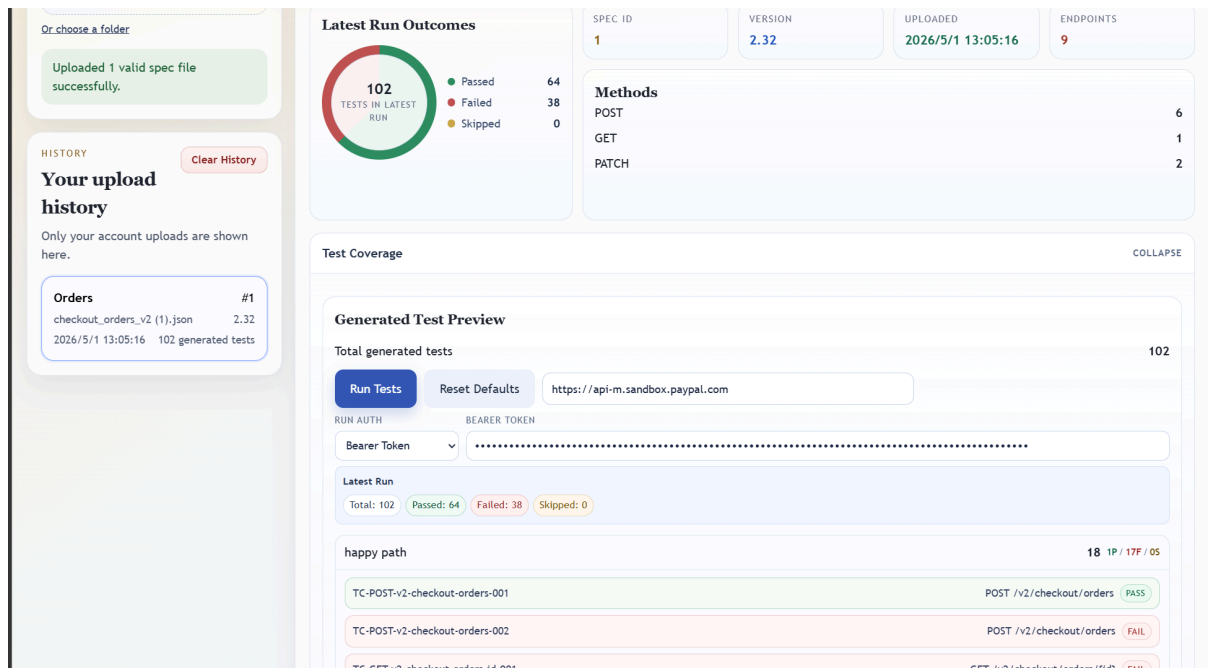
In addition to small sample specifications, ContractGuard was tested against larger real world API specifications: using the full PayPal Checkout API specification and the GitHub REST API specification as examples. These tests were important because they checked whether the full workflow could handle realistic API structures rather than only simplified examples.

For each API specification, the workflow followed the same process: the OpenAPI specification was parsed into the ContractGuard IR, deterministic JSON test cases were generated, the generated suites were executed through the test runner, and the results were recorded as PASS, FAIL, or SKIP.

The GitHub API tests were useful for checking public REST API behavior, authentication handling, and generated test execution against real endpoints. The PayPal Checkout API tests were useful for checking a more business style API with OAuth authentication, sandbox behavior, order creation constraints, and stricter request requirements.

These runs also showed an important limitation of real API testing. Many failed tests were not caused by ContractGuard itself, but by real world execution constraints such as missing authentication, unavailable resource IDs, permission restrictions, unavailable or unrealistic sandbox account details, or endpoints that require specific setup before they can be called successfully.

API	Scope	Results	Main observations
GitHub REST API	Full specification	Generated: 6,273 Executed: 331 Passed: 148 Failed: 183 Manual cutoff: 5,942	Full run was manually stopped due to long execution time. Failures were mainly caused by authentication-sensitive endpoints, insufficient token permissions, enterprise/app-only endpoints, and missing real resources in the test account or repository.
PayPal Checkout API	Full specification	Generated: 102 Executed: 102 Passed: 64 Failed: 38 Skipped / not executed: 0	Failures were mainly caused by real runtime IDs such as order, payment, tracking, and capture IDs. Some endpoints depend on chained workflows, which the current runner does not fully support without human editing or runtime mapping configuration.
GitHub REST API individual user focused subset	7,409 line selected snippet	Generated: 47 Executed: 47 Passed: 32 Failed: 15 Skipped / not executed: 0	Smaller user/repository focused test set. Failures were mainly due to fake or missing owner/repo values and some unavailable token permissions.

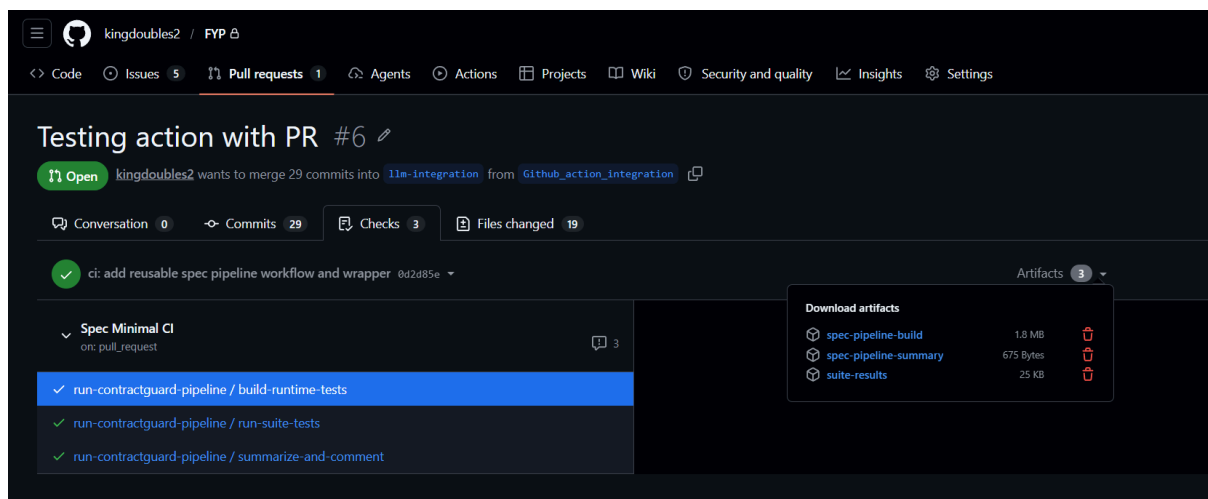


4.2 GitHub Actions Integration Testing with External Developer

The GitHub Actions integration was also tested with two external developers using a separate repository setup. This was useful because it checked whether the CI workflow was understandable and reusable outside the original development environment.

The external developers were asked to configure the workflow with their own OpenAPI specification path, runtime mapping file, and required repository secrets. The expected result was that the GitHub Actions workflow would parse the specification, generate the IR, generate deterministic test cases, execute the generated suite, and expose the run output through CI logs or artifacts.

This test was valuable because it tested more than code correctness. It also tested documentation clarity, workflow usability, repository configuration, secret handling, and whether the pipeline could be reused by someone who was not one of the main developers.



The screenshot displays the GitHub interface for a pull request titled "Testing action with PR #6". The repository is "DoneganB04 / cgtest". The pull request is from "git_act_integration" and contains 38 commits. The workflow "ContractGuard API CI" is shown, with three jobs: "contractguard / build-runtime-tests", "contractguard / run-suite-tests", and "contractguard / summarize-and-comment". The workflow is currently in a failed state, indicated by a red circle.

The "Summary" tab for the workflow "spec-min-ci.yml" is shown, detailing the jobs and their status. The jobs are:

- run-con... / build-runtime-tests (22s)
- run-con... / run-suite-tests (1m 12s)
- r... / summarize-and-commen... (7s)

The "Annotations" section shows three warnings for deprecated actions:

- run-contractguard-pipeline / build-runtime-tests: Node.js 20 actions are deprecated. The following actions are running on Node.js 20 and may not work as expected: actions/checkout@v4, actions/setup-python@v5, actions/upload-artifact@v4. Actions will be forced to run with Node.js 24 by default.
- run-contractguard-pipeline / run-suite-tests: Node.js 20 actions are deprecated. The following actions are running on Node.js 20 and may not work as expected: actions/checkout@v4, actions/download-artifact@v4, actions/setup-python@v5, actions/upload-artifact@v4. Actions will be forced to run with Node.js 24 by default.
- run-contractguard-pipeline / summarize-and-comment: Node.js 20 actions are deprecated. The following actions are running on Node.js 20 and may not work as expected: actions/checkout@v4, actions/download-artifact@v4, actions/upload-artifact@v4. Actions will be forced to run with Node.js 24 by default.

The "Artifacts" section lists three artifacts produced during runtime:

Name	Size	Digest
spec-pipeline-build	1.8 MB	sha256:1f8f36c6852988c4215d9f71c44d1067489f05c192a13a28984f1b45e
spec-pipeline-summary	675 Bytes	sha256:5286f8255a3d5a7a21f1e7b7214711558a66c70f1b4c132ff280743a3b
suite-results	25.9 KB	sha256:f48238fa451f81db119c36c22a093a6d9c3a1f1a338a78c11a51b080c211f882

4.3 Project CI/CD Testing in GitLab

The project also used GitLab CI/CD to test ContractGuard itself during development. This is different from the reusable GitHub Actions integration, because the GitLab pipeline was designed for the project repository rather than for external users.

The GitLab CI/CD pipeline was used to check that the main ContractGuard workflow could still run after code changes. The GitLab CI file runs four stages in sequence:

1. parser check
2. IR extraction
3. test generation
4. test execution.

This helped confirm that the internal workflow from specification input to generated test output was not broken by later changes.

This type of testing was useful because it provided continuous feedback during development. Instead of only running scripts manually, the project could be checked automatically in the repository environment. It also gave evidence that the parser, generator, and runner stages could work together as part of a repeatable pipeline.

The limitation is that GitLab CI/CD mainly checks the project workflow and controlled test environment. It does not fully prove that every live external API test will pass, because APIs such as GitHub and PayPal depend on authentication, permissions, sandbox data, external service state, and resource IDs.

For this CI/CD check, a pass means that the ContractGuard workflow completes successfully and produces the expected outputs. It does not require every generated API test to pass against a live external service, because users may need to adjust test data manually in the frontend or provide runtime mappings for resource dependent endpoints.

Chunyang Wang / ContractGuard / Pipelines

Passed 00:01:10 1 week ago	#86473 generalize JSON Patch defaults using sc... llm-integration 069e7e19 latest branch			
Passed 00:01:19 2 weeks ago	#86265 add safe caps, better allOf merge, and he... llm-integration e82bad13 branch			
Passed 00:01:07 2 weeks ago	#86151 real world Github api spec with chosen e... llm-integration 332f209c branch			
Passed 00:01:16 2 weeks ago	#85925 new real world Paypal api spec llm-integration 1afe6fee branch			
Passed 00:01:17 2 weeks ago	#85921 new real world Paypal api spec llm-integration 39e28bf0 branch			
Warning 00:01:09 2 weeks ago	#85712 individual test runs experimental-scaling 3ce8d82f branch			
Warning 00:01:09 2 weeks ago	#85600 user prompt editing, test run logging AI, e... experimental-scaling 44be2a8e branch			

< Previous Next >

4.4 Manual CLI and Smoke Testing

Alongside the pipeline, the README documents a manual GitHub API flow that is useful for ad hoc system testing. The flow extracts IR from `api.github.com.2026-03-10.yaml`, generates tests, runs them without a token, and then reruns with a bearer token. This is especially useful for a project like ContractGuard because it exposes a very real testing struggle:

authentication can dominate the observed failure mode. The documented unauthenticated result for that sample flow is 3 failed, 8 skipped, which is exactly the kind of behavior that needs to be understood rather than mistaken for a simple backend bug.

The archived smoke run directories under `build/llm_runs_smoke/`, `build/llm_runs_smoke2/`, and `build/llm_runs_smoke3/` also show that lightweight repeat checks were being used during development, even if they were not formalised into a large dedicated regression document.

4.5 Frontend Manual Testing

Area	Expected result	Actual result	Status
Upload API spec	Spec uploads successfully	Spec uploaded and shown in history	Pass
Generate test preview	Generated tests are visible in frontend	Tests displayed by category and endpoint	Pass
Run generated suite	PASS/FAIL/SKIP outcomes are shown	Latest run outcomes displayed	Pass
Base URL and token editing	User can set target API base URL	Base URL field available before run	Pass
History view	Previous uploads and log remain visible	Upload history displayed in sidebar and result logs available	Pass
LLM explanation	LLM should be able to provide explanation based on the actual response body, status code and specific input in each endpoint	Almost all the explanations are consistent with the actual situation.	Pass
Suggestion extra test cases	LLM provide extra suggestion test cases when test fail	can generate some easy and generic value with the required json body. Performance still needs improvement when dealing with real-world and complex data.	Partly Pass

Chat Interface	LLM should be able to answer users questions based on the context and prompt we feed in	Almost all the explanations are consistent with the actual situation.	Pass
LLM models configuration and customise prompt	User can easily change LLM models we integrated to their local or any cloud models and change the prompt we designed for LLM explanations	Works perfect with models in Ollama, OpenAI and Anthropic	Pass

FAILURE DETAILS

Expected status: 200

Actual status: 404

Reason: Not Found

Docs: <https://docs.github.com/rest/repos/repos#get-a-repository>

► Raw response

AI Summary

Suggest Test Cases

AI EXPLANATION

Test input used generic placeholder strings instead of valid existing repository coordinates. Response body explicitly states "Not Found" for the requested path. Confirm test fixture should provision or reference an actual existing repository.

► Prompt sent to model

SUGGESTED EXTRA TEST

Test failed due to invalid repository coordinates. The path parameters used placeholder strings 'standard-text/standard-text' which don't correspond to any existing GitHub repository, resulting in 404. Evidence shows the spec uses 'octocat/Hello-World' as the canonical example repository. Repair by replacing path_params with valid existing repository coordinates while preserving the happy_path intent and expected status 200.

▼ Suggested test JSON

```
{
  "test_id": "TC-GET-repos-owner-repo-001-repair",
  "title": "GET /repos/{owner}/{repo} - Happy path: minimal
valid request returns 200",
  "category": "happy_path",
  "requirement_ref": "GET /repos/{owner}/{repo}",
  "method": "GET",
  "path": "/repos/{owner}/{repo}",
  "priority": "high",
```

Add Suggested Test

ASSISTANT

Chat

Assistant answers are strictly grounded to the selected spec, generated tests, and latest run.

CONTEXT

Selected spec: Orders (#2)

can you summary the top three failure reasons for this run

QWEN3.5:397B-CLOUD SPEC #2 23:41

Based on the 35 failed cases in Run ID 2, the top three failure reasons are:

- 1. Invalid Resource IDs (404 Not Found):**
Approximately 25 failures across GET, PATCH, Authorize, Capture, and Track endpoints returned status 404 with the error code `INVALID_RESOURCE_ID` and message "Specified resource ID does not exist." This indicates the tests are referencing order or tracker IDs that do not exist in the environment.
- 2. Callback Endpoint Errors (404 Not Found):**
All 7 tests for the `/v2/checkout/orders/order-update-callback` endpoint returned status 404. These responses did not include error snippets, distinguishing them from the invalid resource ID errors.
- 3. Business Logic Validation (422 Unprocessable Entity):** Two tests failed with status 422 due to

Ask about this spec's endpoints, generated tests, or latest run...

Scoped to spec #2

Send

(Chat Interface)

ASSISTANT

Close

LLM Settings

Pick your model target and configure custom instruction behavior.

Model Prompt

Choose your model

qwen3.5:397b-cloud

qwen3-coder:latest

kimi-k2.5:cloud

qwen3-coder:480b-cloud

qwen3.5:397b-cloud

API key

Required to load provider models

Model

Enter API key to load models

Label (optional)

Friendly display name

Base URL (optional)

Leave blank for provider default

Add Model

(Configuration LLM models and customise prompts)

TC-GET-repos-owner-repo-001-repair
GET /repos/{owner}/{repo}
PASS

GET /repos/{owner}/{repo} - Happy path: minimal valid request returns 200
Run

Action: Send GET request to /repos/{owner}/{repo} using path params {"owner": "octocat", "repo": "Hello-World"}

INPUT DATA

```

{
  "path_params": {
    "owner": "octocat",
    "repo": "Hello-World"
  },
  "query_params": {},
  "headers": {},
  "body": null
}

```

EXPECTED RESULT

```

{
  "status_code": 200,
  "description": "Validate that GET /repos/{owner}/{repo} keeps 200 after repairing repo to a valid value."
}

```

(Repair test cases generated with LLM)

5. LLM Evaluation During Development

The LLM component needed its own testing track because the question was different. For an endpoint or parser, the main question is usually whether the output is correct. For an explanation model, the more useful questions are:

- did it stay grounded in the supplied evidence?
- did it identify a likely cause rather than hallucinating one?
- did it suggest a sensible next step?
- could it produce a useful follow-up test case when needed?

That is why ContractGuard includes a separate evaluation flow through `config/llm_eval.yaml`, `src/llm_eval/run_llm_eval.py`, the archived runs under `build/llm_runs/`, and the model output archive in `build/backend_llm_report.txt`.

5.1 What the Archived Runs Show

The archived run folders show repeated evaluation rather than one off prompting. An early run, 20260324_160121, recorded:

```
None
Run ID: 20260324_160121
Totals:
- Total: 11
- Passed: 0
- Failed: 3
- Skipped: 8

Model: deepseek-r1:32b
Scores: overall=0.6832 groundedness=0.3333
correctness_proxy=0.3333 relevance=1.0 actionability=1.0
```

One of the explanations from that run reads:

```
None
TC-GET-user-001 (fail): Likely cause: The request was made
without proper authentication.
Why likely: The test did not include any authentication
headers, leading to a 401 Unauthorized response.
Check next: Ensure that the Authorization header with a valid
Bearer token is included in the request.
```

This is a good example of the benefit of the evaluation setup. The model did not jump straight to blaming application logic; it identified that the failure was dominated by missing authentication, which is exactly the kind of distinction the team needed during development.

A later run, 20260324_192301, showed a broader and more demanding set of cases:

```
None
Run ID: 20260324_192301
Totals:
- Total: 11
- Passed: 4
- Failed: 7
- Skipped: 0

Model: deepseek-r1:32b
Scores: overall=0.8271 groundedness=0.25 correctness_proxy=1.0
relevance=1.0 actionability=1.0
```


One of the later failure summaries was:

None

TC-GET-repos-owner-repo-001 (fail): Likely cause: The observed status code did not match the expected contract outcome for this case.

Why likely: Expected 200 but observed 404, indicating behavior mismatch for this specific test intent.

Check next: Inspect endpoint specific server behavior and whether this negative-case expectation is still correct.

This is useful for a different reason. By this point the evaluation is not only catching obvious auth walls; it is also surfacing situations where the contract expectation itself may need to be revisited.

5.2 Suggested-Test Generation

The saved LLM output log also shows that the system was not limited to writing explanations. It was used to propose follow-up tests based on the failure evidence. One archived entry in `build/backend_llm_report.txt` contains the following suggestion for TC-GET-repos-owner-repo-001:

None

```
suggested_test_case={"test_id":
"TC-GET-repos-owner-repo-001-FOLLOWUP-LLM-01", ...
"expected_result": {"status_code": 404, "description":
"Validate the observed behavior path currently returning
404."}}
```

That is a practical testing benefit. Instead of stopping at “the model generated an explanation”, the workflow tries to turn that explanation into the next concrete thing worth checking.

5.3 Benefits, Struggles, and Accommodations

The main benefit of this LLM evaluation work is that it made the model layer much less anecdotal. Runs were timestamped, cases were archived, and outputs were scored across dimensions such as groundedness, correctness proxy, relevance, actionability, conciseness, and contextual awareness.

At the same time, the runs also show the harder parts clearly. Groundedness was noticeably weaker than relevance or actionability in the examples above. That is useful information in itself: it means the model was often able to produce something helpful, but keeping it tightly anchored to the supplied evidence remained the harder problem.

The project accommodated this in a few ways:

- evaluation focused on failed cases rather than trying to score everything at once
- evidence bundles were saved alongside the model output
- output length and retries were constrained in configuration
- deterministic fallbacks and structured contracts were tested in the backend suite

6. Summary of What Was Tested

Area	What was tested	What it added	Current limitation
Backend services	provider clients, LLM endpoints, persistence, merge logic, logistics views, explanation and suggested test flows	fast feedback on the core Python behavior, with stable automated reruns	does not cover every deployment specific condition
Parser stage	Spec parsing and IR generation	Showed that ContractGuard can parse real world OpenAPI specifications and convert them into the internal representation used by the rest of the workflow.	Performance cannot be guaranteed for extremely large specifications that contain deeply nested schemas, complex references, or multiple layers of recursion.
Test generation and runner	generated JSON suites, auth-aware execution, structured result recording	shows the project works against real APIs rather than only mocks	external APIs and auth conditions can distort the failure mode
Project CI/CD testing in GitLab	Parser check, IR extraction, test generation, and test execution in the project repository	Provided continuous feedback during development and checked that the internal workflow still ran after code changes	Does not prove every live external API test will pass because external APIs depend on auth, permissions, sandbox data, and resource state
LLM evaluation	archived explanation runs, score summaries, suggested test generation	helped compare model behavior and make the explanation layer more systematic	model usefulness still depends on grounding quality

Area	What was tested	What it added	Current limitation
Frontend manual testing	Upload flow, generated test preview, run results, base URL/token editing, history view, LLM explanation, chat interface, and LLM settings	Shows that the main user facing workflow was manually checked through the UI	No automated frontend test suite was implemented
Full GitHub REST API testing	Can generate and execute test suites against real APIs	Showed ContractGuard can work with a large public API and handle auth-sensitive endpoints	Some failures depend on token permission, enterprise/app-only endpoints, and missing real resources in the test account or repository.
Full PayPal Checkout API testing	Full PayPal Checkout spec tested through sandbox/API execution	Showed the system can handle OAuth-based business APIs and realistic request constraints	Sandbox setup, OAuth state, chained workflow support and required resource IDs can affect results
External GitHub Actions testing	Two external developers tested the reusable GitHub Actions integration	Checked usability outside the project team and proved CI integration is not only local	Depends on correct repo secrets, paths, and workflow configuration

7. Limitations and Remaining Gaps

The main remaining limitation on the frontend side is that testing was manual rather than automated. The upload flow, generated test preview, run result display, history view, LLM explanation panel, chat interface, and LLM settings were checked manually, but no automated frontend test suite was implemented.

There is also no formal coverage report in the project at the moment. Therefore, the strength of testing is shown through passing backend tests, parser checks, CI/CD pipeline runs, real API execution results, frontend manual testing evidence, and archived LLM evaluation outputs rather than through a single coverage percentage.

The parser check is useful, but some of the parser validation is still closer to smoke testing than deep assertion-based testing. The system was able to handle large real-world specifications such as the GitHub REST API and PayPal Checkout API, but performance cannot be fully guaranteed for extremely large specifications with deeply nested schemas, complex references, or multiple layers of recursion.

Real API testing also has practical limits. Some failed GitHub and PayPal test cases were caused by authentication requirements, token permissions, sandbox state, missing real resource IDs, or endpoints that require chained workflows. These failures are still useful because they show real-world API constraints, but they should not all be interpreted as defects in ContractGuard itself.

The LLM evaluation work is valuable, but it should not be treated as proof that the whole application is correct. It mainly evaluates whether the explanation and suggested-test layer is grounded, relevant, and useful. Model output can still vary depending on the selected provider, prompt, evidence quality, and complexity of the failed case.

Finally, although GitLab CI/CD and GitHub Actions integration testing provide useful evidence, they cover different purposes. GitLab CI/CD mainly tests the ContractGuard project workflow during development, while the reusable GitHub Actions workflow tests whether other developers can run ContractGuard in their own repositories. Both depend on correct configuration, secrets, paths, and runtime mappings.